

B ve B+ Ağaçlarının Günümüzde Kullanım Alanları

Veri Tabanlarından Dosya Sistemlerine — Bilgiyi Organize Etme Sanatı

FATMA DİCLE

İNÖNÜ ÜNİVERSİTESİ — YAZILIM MÜHENDİSLİĞİ

ÖĞRENCİ NO: 02240224172

Teorik Temeller: B-Tree ve B+ Tree Nedir?

B-Tree ve B+ Tree, büyük veri kümelerini verimli biçimde yönetmek için tasarlanmış, dengeli arama ağacı veri yapılarıdır. Her ikisi de arama, ekleme ve silme işlemlerini $O(\log n)$ zaman karmaşıklığında gerçekleştirir; ancak veriyi saklama biçimleri birbirinden ayrılır.

B-Tree

Verileri sıralı tutan, her düğümünde hem anahtar hem de veri barındıran klasik yapıdır. İç düğümler de veri depolar.

- Tüm düğümlerde veri saklanır
- Arama herhangi bir seviyede tamamlanabilir
- Aralık sorguları için ek tarama gerekebilir

B+ Tree

B-Tree'nin optimize edilmiş versiyonudur. Tüm veriler yalnızca yaprak düğümlerinde saklanır; iç düğümler yalnızca yönlendirme anahtarları içerir.

- Veriler yalnızca yapraklarda tutulur
- Yaprak düğümler birbirine zincirlenmiştir
- Aralık sorguları son derece hızlıdır



Günlük Hayattan Benzetme: Hastane Arşivi

Milyonlarca hasta dosyasının soyadına göre sıralı klasörlerde durduğu devasa bir hastane arşivini hayal edin. Bu arşiv, B-Tree ve B+ Tree arasındaki farkı somutlaştırmak için mükemmel bir analogi sunar.



B-Tree Arşivi

Hastanede tüm dolaplarda veri vardır. Belirli bir hastayı ararken arama doğrudan ilgili dolap rafında sona erebilir. Ancak **"A'dan F'ye kadar tüm hastaları listele"** gibi bir aralık sorgusunda tüm arşivi dolaşmak gerekebilir — veri her yerde dağınık durumdadır.

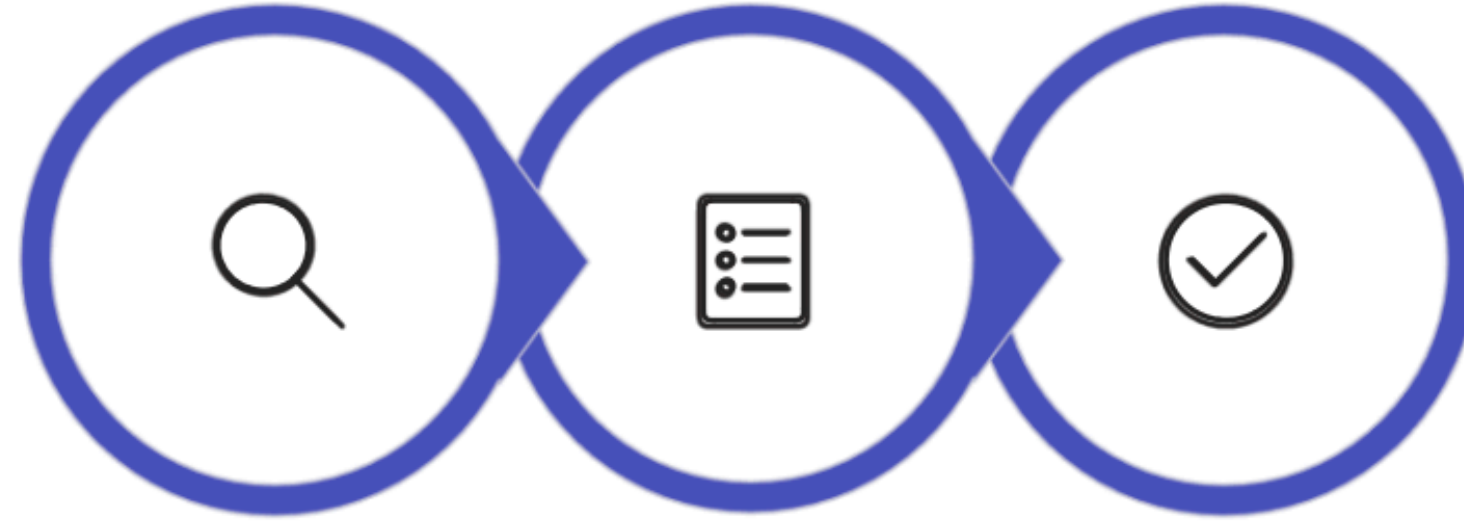


B+ Tree Arşivi

İç düğümler yalnızca **"A-F"** veya **"G-M"** yazan bölücü etiketlerdir, sizi doğru rafa yönlendirir. Asıl dosyalar ise rafın en altındaki çekmecelerde zincir gibi birbirine bağlıdır. Aralık sorgusu = başlangıç noktasını bul, sonra zinciri takip et.

Bu Anolojinin Somut Faydası: Aralık Sorguları

"Ahmet'ten Ayşe'ye kadar tüm hastaları listele" — bu sıradan bir SQL sorgusudur. B+ Tree bu sorguyu nasıl ele alır? İşte adım adım süreç:



Başlangıç Bul

Sıralı Tarama

Sonuç
Döndür

B+ Tree Avantajı

Zincirlenmiş yaprak düğümler sayesinde, başlangıç noktası bulunduğundan sonra tüm aralık **tek bir sıralı okuma** ile taranır. Disk başlığı geri gitmez, arama yeniden başlamaz.

B-Tree'de Sorun

B-Tree'de aynı aralık sorgusu, iç düğümlere tekrar tekrar dönmeyi gerektirebilir. Her veri noktası farklı bir ağaç seviyesinde olabileceğinden **disk I/O maliyeti** artar.



Sonuç: B+ Tree, aralık sorgularında B-Tree'ye kıyasla çok daha az disk erişimi ile aynı sonucu üretir.



Modern Sistemlerde Neden Tercih Edilir?

B+ Tree'nin beş dekadı aşan yaşına rağmen modern yazılım altyapısının baskın veri yapısı olmaya devam etmesinin üç temel nedeni vardır:



Disk I/O Verimliliği

Blok tabanlı okuma mimarisiyle uyum içinde çalışır. Her düğüm, bir disk bloğuna sığacak şekilde boyutlandırılır; bu da disk erişim sayısını minimuma indirir.



Garantili Denge

Her ekleme ve silme işleminden sonra ağaç kendini yeniden dengeler. Bu sayede **en kötü durumda bile $O(\log N)$** performans garantisi sağlanır.



Önbellek Dostu Yapı

Yaprak düğümlerde veri yakınlığı (locality) sağlanır. Ardışık okumalar CPU önbelleğini verimli kullanır; özellikle NVMe SSD'lerde belirgin hız kazanımı gözlemlenir.

Veri Tabanı Motorlarındaki Kullanımı

Dünyanın en yaygın kullanılan veri tabanı sistemlerinin tamamı — ilişkisel ya da NoSQL — B-Tree ailesini indeks yapısı olarak benimser.



MySQL / InnoDB

B+ Tree, birincil anahtar ve tüm ikincil indeksler için kullanılır. Tablo verisi doğrudan yaprak düğümlerinde **clustered index** olarak saklanır.



PostgreSQL

Varsayılan indeks türü B-Tree'dir. **WHERE** koşulları ve **ORDER BY** sorgularında otomatik devreye girer, ek yapılandırma gerektirmez.



MongoDB

WiredTiger depolama motoru B-Tree tabanlı indeksleme kullanır. NoSQL belgelerinde bile bu yapı, sorgu performansının temel taşıdır.

Dosya Sistemlerindeki Yeri

B-Tree yapıları yalnızca veri tabanlarında değil, işletim sistemlerinin depolama katmanında da kritik rol oynar. Her dosya açma işleminde, arka planda bir B-Tree traversal gerçekleşir.

İşletim Sistemi Desteği

→ Windows — NTFS

Master File Table (MFT), B-Tree yapısıyla düzenlenir. Her dizin girişi bu ağaç üzerinden bulunur.

→ Linux — ext4

Dizin girişleri HTree (B-Tree türevi) ile indekslenir. Büyük dizinlerde performans kazanımı kritiktir.

→ macOS — APFS

Dosya meta verisi B-Tree ağaçlarında tutulur; anlık görüntü (snapshot) desteği de bu yapı üzerine inşa edilmiştir.

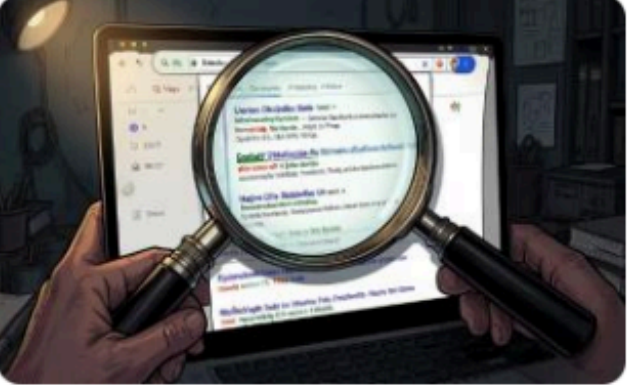
Doğrudan Blok Erişimi

Dosya sistemi, her okuma/yazma işleminde bloğun fiziksel disk adresini B-Tree indeksi üzerinden milisaniyeler içinde bulur. Bu olmadan, milyonlarca dosya içinde arama lineer zaman alırdı.

i Bir SSD üzerinde 1 milyon dosya arasında arama: B-Tree ile ~20 karşılaştırma, lineer tarama ile ~500.000 karşılaştırma.

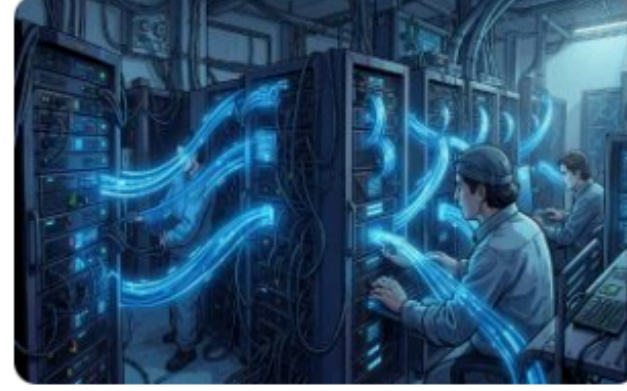
Arama Motorları ve Büyük Veri

B-Tree ailesi, internet ölçeğindeki sistemlerde de mimarinin temelini oluşturur. Milyarlarca belge veya satır arasında milisaniyeler içinde yanıt üretme kapasitesi, bu yapı sayesinde mümkün olmaktadır.



Elasticsearch

Ters indeks yapılarını B-Tree'lerle destekleyerek milyarlarca belgede **milisaniye** yanıt süreleri sağlar. Tam metin arama + yapısal sorgu birleşiminde kritiktir.



Hadoop / HBase

Bölge sunucularında indeks yapısı olarak B-Tree türevleri kullanılır. Petabayt ölçeğinde veriyi bölümlere ayırarak yönetir.



AWS DynamoDB

Dağıtık ortamda B-Tree ilkelerini ölçeklenebilir yapılarla birleştirir. Küresel çapta düşük gecikme ile yüksek erişilebilirlik sağlar.

Akademik Değerlendirme: Neden Bu Kadar Önemlidir?

50+

Yıllık Aktif Kullanım

B-Tree ailesi 1972'den bu yana üretim sistemlerinde kesintisiz kullanılmaktadır.

$O(\log N)$

Garantili Karmaşıklık

En kötü senaryoda bile logaritmik performans — büyük veri çağının olmazsa olmazı.

100%

Ana DB Motoru Desteği

MySQL, PostgreSQL, MongoDB, Oracle, SQLite — tümü B-Tree tabanlı indeks kullanır.

Büyük Veri Çağının Temeli

Günde milyarlarca işlem yapan e-ticaret ve bankacılık sistemleri B+ Tree indekslerine dayanır. Amazon, Google, Meta gibi teknoloji devlerinin altyapısı bu yapı üzerine inşa edilmiştir.

Donanım ile Mükemmel Uyum

HDD'lerin blok tabanlı mantığı için tasarlandı; SSD ve NVMe'nin yüksek paralellik kapasitesine de uyarlandı. Depolama teknolojisi değişse de B+ Tree adapte olmayı başarmıştır.

SQL veya NoSQL fark etmeksizin, tüm modern veri tabanlarında **standart indeks altyapısı** haline gelmiştir.



Gelecekte Kullanılacak mı? — Kesinlikle Evet

B+ Tree, 50 yıldır hayatta ve yakın gelecekte emekli olmayı planlamıyor. Güçlü adaptasyon kapasitesi, onu her yeni nesil teknolojiye uyumlu kılmaktadır.

- 1 — 1972 — Doğuş**
Rudolf Bayer ve Ed McCreight, B-Tree'yi IBM araştırma laboratuvarında geliştirdi.
- 2 — 1990'lar — Standartlaşma**
İlişkisel veri tabanlarının patlama döneminde fiili standart indeks yapısı haline geldi.
- 3 — 2010'lar — Dağıtık Sistemler**
NoSQL ve büyük veri platformlarına uyarlandı; Bw-Tree gibi yeni varyantlar doğdu.
- 4 — Günümüz ve Sonrası**
NVMe SSD optimizasyonu, B-Epsilon Tree ve dağıtık varyantlarla geleceğe taşınmaktadır.

Hibrit yaklaşımlar güçlense de, B+ Tree'nin dengeli okuma/yazma performansı onu **vazgeçilmez** kılmaya devam edecektir. Bu yapıyı anlamak, her yazılım mühendisi için temel bir yetkinliktir.